

## ADVANCE FILE OPERATIONS

### Race Condition

1. Write a C program where two processes simultaneously write to the same file. Demonstrate the race condition and suggest a solution using file locks.

### PROGRAM:

```
/*
Program to implement race condition
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : race.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

//Functions to lock and unlock the file
void lock_file(int fd)
{
    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;
    fcntl(fd, F_SETLKW, &lock);
}

void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

int main()
```

```

{
    int fd;
    pid_t pid;
    char buffer[100];

    fd = open("race.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    for(int i=0;i<5;i++)
    {
        lock_file(fd);

        if(pid==0)
            sprintf(buffer,"Child writing %d\n",i);
        else
            sprintf(buffer,"Parent writing %d\n",i);

        write(fd,buffer,strlen(buffer));

        unlock_file(fd);

        sleep(1);
    }

    close(fd);
}

```

```

/*
Program to prevent file locking
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : race.c
*/

```

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

```

```

//Functions to lock and unlock the file
void lock_file(int fd)

```

```

{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLKW, &lock);
}

//Functions to lock and unlock the file
void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

int main()
{
    int fd;
    pid_t pid;
    char buffer[50];

    fd = open("race.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    for(int i = 0; i < 5; i++)
    {
        lock_file(fd);

        if(pid == 0)
            sprintf(buffer, "Child process writing %d\n", i);
        else
            sprintf(buffer, "Parent process writing %d\n", i);
    }
}

```

```

        write(fd, buffer, strlen(buffer));

        unlock_file(fd);

        usleep(100000);
    }

    close(fd);
    return 0;
}

```

## OUTPUT:

The screenshot shows a WSL terminal window with the following commands and output:

```

mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc race
.c -o race
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./race
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc lock
.c -o lock
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./lock
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$

```

Below the terminal, a text editor window shows the content of 'race.txt':

```

Parent writing 0
Child writing 0
Parent writing 1
Child writing 1
Parent writing 2
Child writing 2
Parent writing 3
Child writing 3
Parent writing 4
Child writing 4

```

## Atomicity

1. Write a C program that appends data to a file from multiple processes. How do you ensure the append operation is atomic?

## PROGRAM:

```

/*
Program to implement atomic log file
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : atomic.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

```

```

int main()
{
    int fd;
    pid_t pid;
    char buffer[100];

    fd = open("atomic_log.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if(fd < 0)
    {
        perror("open");
        exit(1);
    }

    pid = fork();

    for(int i = 0; i < 5; i++)
    {
        if(pid == 0)
            sprintf(buffer, "Child process %d writing
line %d\n", getpid(), i);
        else
            sprintf(buffer, "Parent process %d writing
line %d\n", getpid(), i);

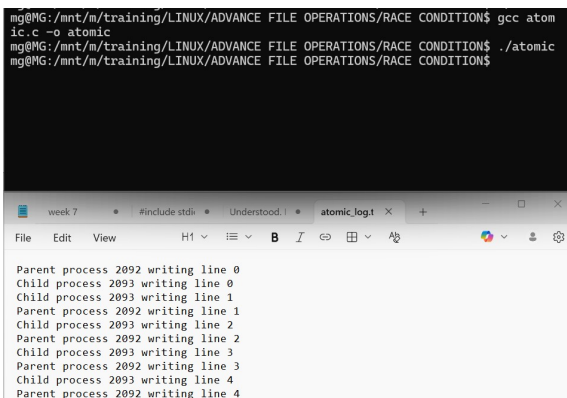
        write(fd, buffer, strlen(buffer));

        sleep(1);
    }

    close(fd);
    return 0;
}

```

## OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITIONS$ gcc atom
ic.c -o atomic
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITIONS$ ./atomic
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITIONS$

```

Parent process 2092 writing line 0  
Child process 2093 writing line 0  
Child process 2093 writing line 1  
Parent process 2092 writing line 1  
Child process 2093 writing line 2  
Parent process 2092 writing line 2  
Child process 2093 writing line 3  
Parent process 2092 writing line 3  
Child process 2093 writing line 4  
Parent process 2092 writing line 4

2. Demonstrate atomic read-modify-write on a shared file using system calls. What happens if atomicity is not guaranteed?

**PROGRAM:**

```
/*
Program to read write modify in atomic mode
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : rwm.c
*/

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

void lock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLKW, &lock);
}

void unlock_file(int fd)
{
    struct flock lock;

    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    fcntl(fd, F_SETLK, &lock);
}

int main()
{
    int fd;
    int value;
    pid_t pid;

    fd = open("counter.txt", O_RDWR | O_CREAT, 0644);
```

```

if(fd < 0)
{
    perror("open");
    exit(1);
}

pid = fork();

for(int i = 0; i < 5; i++)
{
    lock_file(fd);

    lseek(fd, 0, SEEK_SET);
    read(fd, &value, sizeof(int));

    value++;

    lseek(fd, 0, SEEK_SET);
    write(fd, &value, sizeof(int));

    printf("Process %d updated counter to %d\n", getpid(),
value);

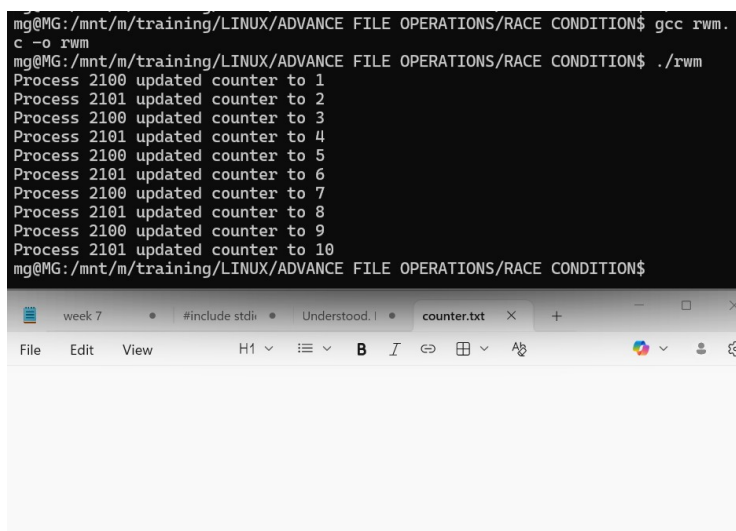
    unlock_file(fd);

    sleep(1);
}

close(fd);
return 0;
}

```

## OUTPUT:



```

mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc rwm.c -o rwm
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./rwm
Process 2100 updated counter to 1
Process 2101 updated counter to 2
Process 2100 updated counter to 3
Process 2101 updated counter to 4
Process 2100 updated counter to 5
Process 2101 updated counter to 6
Process 2100 updated counter to 7
Process 2101 updated counter to 8
Process 2100 updated counter to 9
Process 2101 updated counter to 10
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$

```

## Pre-Emptive and Non Pre-Emptive Concept

1. Write a C program using threads to show how pre-emptive scheduling can cause unexpected results when accessing shared data.

### PROGRAM:

```
/*
Program to demonstrate the preemption of threads
Author   : MUTHUGANESH S
Date     : 07/02/2026
Filename : pre.c
*/

#include <stdio.h>
#include <pthread.h>

int counter = 0;

//Function to increment the counter
void *increment(void *arg)
{
    for(int i = 0; i < 1000000; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    //Declare two threads
    pthread_t t1, t2;

    //Create two threads
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    //Join the threads
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value: %d\n", counter);

    return 0;
}
```



## OUTPUT:

```
Process 1101 updated counter to 10
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc pre.c -o pre
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./pre
Final Counter Value: 1163685
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ |
```

## File Descriptor Table and Open File Table

1. Write a C program to open multiple files and display their file descriptors. Explain the relationship between the process's file descriptor table and the system-wide open file table.

## PROGRAM:

```
/*
Program to create 3 files
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : ThreeFiles.c
*/

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    //Declaring file descriptors
    int fd1, fd2, fd3;

    //Creating 3 files
    fd1 = open("file1.txt", O_CREAT | O_RDWR, 0644);
    fd2 = open("file2.txt", O_CREAT | O_RDWR, 0644);
    fd3 = open("file3.txt", O_CREAT | O_RDWR, 0644);

    if(fd1 < 0 || fd2 < 0 || fd3 < 0)
    {
        perror("File open error");
        return 1;
    }

    printf("File Descriptor for file1.txt : %d\n", fd1);
    printf("File Descriptor for file2.txt : %d\n", fd2);
    printf("File Descriptor for file3.txt : %d\n", fd3);

    close(fd1);
```

```

        close(fd2);
        close(fd3);

        return 0;
    }

```

### **OUTPUT:**

```

mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc ThreeFiles.c -o ThreeFiles
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./ThreeFiles
File Descriptor for file1.txt : 3
File Descriptor for file2.txt : 4
File Descriptor for file3.txt : 5
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ |

```

2. Demonstrate how closing a file descriptor affects the file descriptor table and open file table. What happens if multiple descriptors refer to the same file?

### **PROGRAM:**

```

/*
Program to create a file and duplicate the file descriptor
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : dup.c
*/

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    //Declaring file descriptors
    int fd1, fd2;

    //Creating a file
    fd1 = open("test.txt", O_CREAT | O_RDWR, 0644);

    //Duplicating the file
    fd2 = dup(fd1);

    //Print the file descriptors
    printf("Original FD: %d\n", fd1);
    printf("Duplicate FD: %d\n", fd2);
}

```

```

    close(fd1);

    printf("Closed fd1\n");

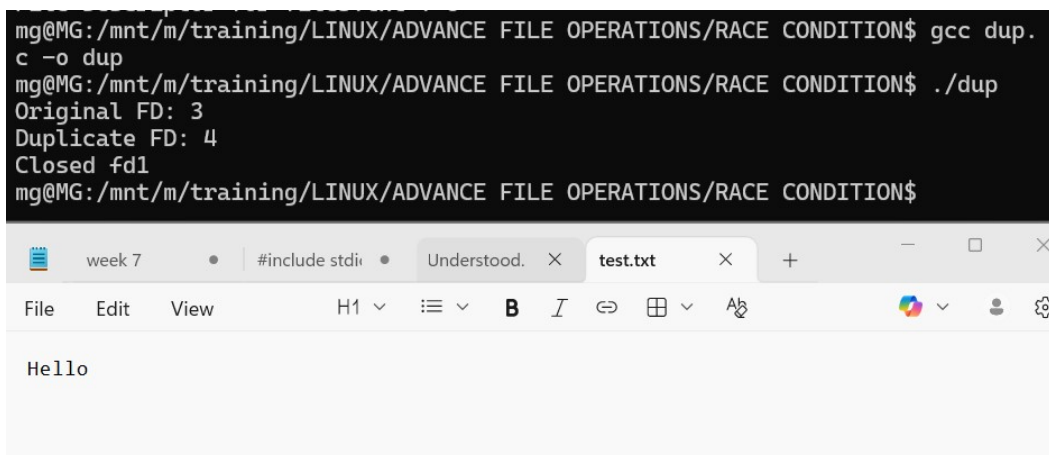
    write(fd2, "Hello\n", 6);

    close(fd2);

    return 0;
}

```

## **OUTPUT:**



The screenshot shows a terminal window with the following commands and output:

```

mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc dup.c -o dup
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./dup
Original FD: 3
Duplicate FD: 4
Closed fd1
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$

```

Below the terminal window is a text editor window titled 'test.txt' showing the output of the program:

```

Hello

```

## **Duplicating File Descriptor - dup() System Call**

1. Write a C program that uses dup() to duplicate a file descriptor and show that both descriptors refer to the same open file description.

## **PROGRAM:**

```

/*
Program to create a file and duplicate the file descriptor
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : dup.c
*/

//header files
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{

```

```

int fd, fd_dup;

//Creating a file
fd = open("dup_example.txt", O_CREAT | O_RDWR | O_TRUNC,
0644);

if(fd < 0)
{
    perror("open failed");
    return 1;
}

//Duplicating the file
fd_dup = dup(fd);

printf("Original FD : %d\n", fd);
printf("Duplicated FD : %d\n", fd_dup);

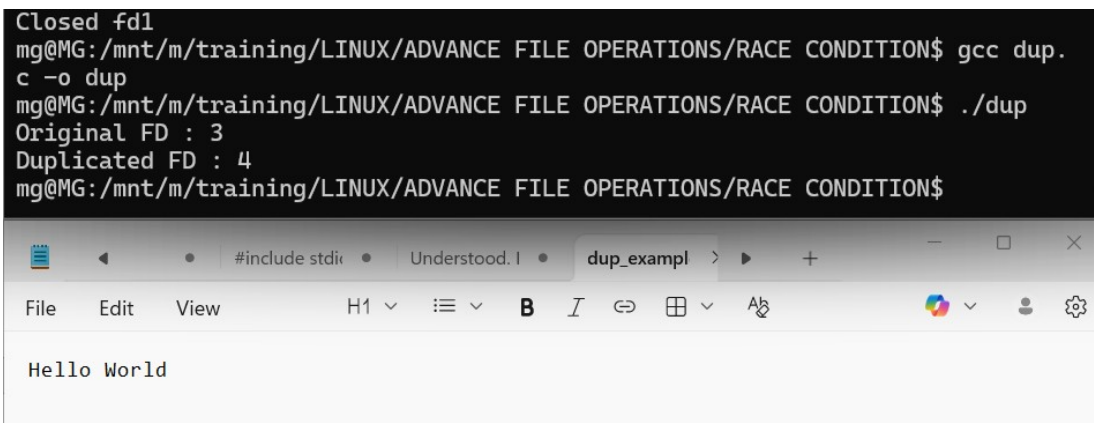
write(fd, "Hello ", 6);
write(fd_dup, "World\n", 6);

close(fd);
close(fd_dup);

return 0;
}

```

## OUTPUT:



The screenshot shows a terminal window at the top with the following commands and output:

```

Closed fd1
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc dup.c -o dup
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./dup
Original FD : 3
Duplicated FD : 4
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$

```

Below the terminal is a code editor window titled 'dup\_exempl'. The editor shows the source code of the program, and the output of the program is displayed in the lower pane: 'Hello World'.

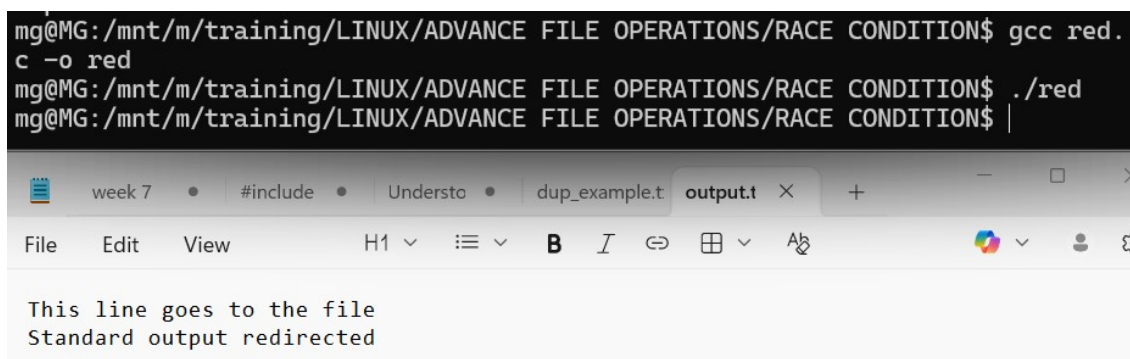
2. Implement a program that redirects standard output to a file using dup2(). Verify that output is written to the file instead of the console.

### **PROGRAM:**

```
/*
Program to create a file and redirect the standard output to
the file
Author   : MUTHUGANESH S
Date     : 05/02/2026
Filename : red.c
*/

//Header files
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    //Creating a file
    fd = open("output.txt", O_CREAT | O_WRONLY | O_TRUNC,
0644);
    //Checking if the file is created
    if(fd < 0)
    {
        perror("open failed");
        return 1;
    }
    dup2(fd, STDOUT_FILENO);
    printf("This line goes to the file\n");
    printf("Standard output redirected\n");
    close(fd);
    return 0;
}
```

### **OUTPUT:**



The screenshot shows a terminal window with the following commands and output:

```
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ gcc red.c -o red
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$ ./red
mg@MG:/mnt/m/training/LINUX/ADVANCE FILE OPERATIONS/RACE CONDITION$
```

Below the terminal, a text editor window is open, displaying the output of the program:

```
This line goes to the file
Standard output redirected
```

